Lecture 14: IP Addressing, CIDR, and Subnetting

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

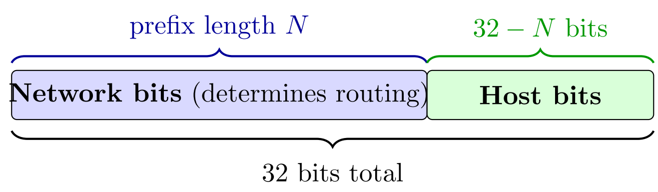
1. Explain why an IP address is divided into a network portion and a host portion, and what that split enables for hierarchical routing
2. Describe classful addressing (Class A/B/C/D/E) and explain why it was abandoned
3. Interpret CIDR prefix notation (e.g., `192.168.1.0/24`) and relate it to a subnet mask
4. Compute the network address, broadcast address, and usable host range for a given CIDR prefix
5. Divide a given address block into equal-sized subnets using a longer prefix length
6. Determine whether two hosts are in the same subnet given their addresses and prefix lengths
7. Identify the major special-purpose IPv4 address ranges (RFC 1918, loopback, link-local, multicast, CGN)
8. Describe the IANA → RIR → ISP → organization address allocation hierarchy and explain why IPv4 exhaustion occurred
9. Use Python's `ipaddress` standard-library module to perform address and subnet arithmetic

1 The Network/Host Split: The Core Mental Model

Lecture 13 introduced CIDR notation (`10.1.2.0/24`, `192.168.0.0/16`) without fully explaining the underlying logic. This lecture provides that foundation.

An IPv4 address is 32 bits. Those 32 bits encode **two distinct pieces of information** packed together:

1. **Network identifier** — which network the host belongs to
2. **Host identifier** — which specific device within that network

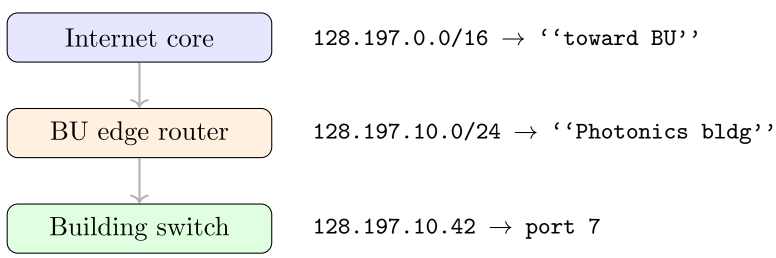


The **prefix length** (`/N`) specifies how many of the 32 bits are network bits. Everything after bit N is the host portion.

Why This Split Matters: Hierarchical Routing

This encoding is the reason the internet can scale. Consider the alternative: if every router needed an entry for every host, the global routing table would have billions of entries—one per device. That is not feasible.

Instead, the network bits enable **aggregation**. A router in Japan does not need to know about individual machines at BU. It only needs: "for anything whose network bits match BU's block, forward toward BU." Only the routers *at* BU need finer-grained entries for individual subnets and hosts. The postal-code analogy makes this intuitive: a sorting facility in Tokyo does not need every street address in Boston—it just needs to know "02215 → Boston."



Each level knows only what it needs to. Prefix length controls the granularity.

Key mental model for prefix lengths:

- **/0** — matches *everything*; the default route
- **/8** — large network; 16 million host addresses; appropriate for ISPs
- **/24** — standard LAN segment; 256 addresses, 254 usable
- **/30** — point-to-point link; 4 addresses, 2 usable
- **/32** — a single host; the "network" contains exactly one address

2 IPv4 Address Structure and Classful Addressing

The 32-Bit Address

IPv4 uses a 32-bit address space: $2^{32} \approx 4.3$ billion addresses total.[1]

Dotted-decimal notation: the 32 bits are split into four 8-bit octets, each written as a decimal number 0–255, separated by dots:

[1] Wikipedia: IPv4 — address structure, packet format, dotted-decimal notation, and the 32-bit address space.

Binary:	`10000000`	·	`11000101`	·	`00001010`	·	`00101010`
Decimal:	128	·	197	·	10	·	42

Conversions to know: `0x00` = 0; `0xFF` = 255; `0x80` = 128; `0xC0` = 192; `0xA0` = 160. The address `128.197.10.42` lies in BU's assigned address space (`128.197.0.0/16`). Dotted-decimal is purely a human convenience; router hardware works on bits.

Classful Addressing (1981–1993)

When IPv4 was defined (RFC 791, 1981),[2] the prefix length was not written explicitly. Instead, the **first few bits** of the address indicated the "class," and the class determined the prefix length:

Class	Leading bits	Prefix	Networks	Hosts/network	Range
A	`0`	/8	128	16,777,214	0.0.0.0 – 127.255.255.255
B	`10`	/16	16,384	65,534	128.0.0.0 – 191.255.255.255
C	`110`	/24	2,097,152	254	192.0.0.0 – 223.255.255.255
D	`1110`	—	Multicast		224.0.0.0 – 239.255.255.255
E	`1111`	—	Reserved		240.0.0.0 – 255.255.255.255

Reading an address: if the first octet is 128–191, the address is Class B (leading `10`), prefix is /16, and the first two octets identify the network. `128.197.10.42` → Class B, network `128.197.0.0`, host `.10.42`. Note: BU's `/16` block is a Class B allocation dating from the classful era.

Why Classful Addressing Failed

By the early 1990s, classful addressing had created two serious problems.

Problem 1: Waste at every scale. The binary choice — Class B (65 K hosts) or Class C (254 hosts) — matched almost no real organization's needs. A university with 5,000 students received a Class B and immediately wasted 60,000 addresses. A medium-sized company with 2,000 employees needed a Class B but received eight Class C blocks instead, which also wasted space. By 1993, roughly half of the assigned IPv4 address space was sitting unused inside organizations with overly large allocations.

Problem 2: Routing table explosion. A company with eight Class C blocks had eight separate entries in the global routing table. There was no way to aggregate them: classful rules did not allow prefixes shorter than /24 for Class C addresses. By 1993, the global routing table was growing exponentially and router hardware was struggling to keep up.

The core mismatch:

- Small company (50 hosts): Class C is adequate (✓)
- Medium company (2,000 hosts): too big for Class C, too small for Class B → one /16 allocated, 63,000 addresses wasted
- Large ISP: Class A is adequate (✓), but millions of addresses may sit unused

CIDR was the fix.

[2] Wikipedia: Classful network — Class A/B/C/D/E, leading-bit encoding, historical allocation, and why it was abandoned in 1993.

3 CIDR: Classless Inter-Domain Routing

The CIDR Solution (RFC 1519, 1993)

CIDR[3] made three changes to the classful model:

1. **Any prefix length is valid.** The split point is no longer dictated by address class. You can have a /19, a /22, a /27 — whatever fits the network.
2. **Prefix length is written explicitly.** `192.168.1.0/24`, not "Class C, therefore /24."
3. **Aggregation is first-class.** Adjacent blocks with a common prefix can be represented by a single shorter-prefix entry. This is called **supernetting** or **route aggregation**.

Subnet Masks

A CIDR prefix is equivalent to a **subnet mask**:[4] a 32-bit number with N ones followed by $(32-N)$ zeros.

Prefix	Binary mask	Dotted-decimal
/24	`11111111.11111111.11111111.00000000`	`255.255.255.0`
/20	`11111111.11111111.11110000.00000000`	`255.255.240.0`
/26	`11111111.11111111.11111111.11000000`	`255.255.255.192`
/30	`11111111.11111111.11111111.11111100`	`255.255.255.252`

Given an address and mask, the **network address** is the bitwise AND of the two:

```
  192.168.1.75  =  11000000.10101000.00000001.01001011
& 255.255.255.0 =  11111111.11111111.11111111.00000000
                   -----------------------------------
                   11000000.10101000.00000001.00000000  =  192.168.1.0
```

The **broadcast address** has all host bits set to 1. For `192.168.1.0/24`: broadcast is `192.168.1.255`.

Usable host range: network address +1 through broadcast address −1. For `192.168.1.0/24`: `.1` through `.254` = 254 usable hosts.

General formula for a `/N` prefix:

- Total addresses: $2^{(32-N)}$
- Usable hosts: $2^{(32-N)} - 2$ (subtract network address and broadcast)

[3] Wikipedia: Classless Inter-Domain Routing — CIDR, prefix notation, route aggregation, and RFC 1519.
[4] Wikipedia: Subnet mask — mask representation, bitwise AND with address, prefix-length equivalence.

Prefix	Total	Usable	Typical use
/30	4	2	Point-to-point links
/29	8	6	Very small LAN
/28	16	14	Small office
/27	32	30	Small office
/26	64	62	Small office
/25	128	126	Medium office
/24	256	254	Standard LAN segment
/23	512	510	Medium campus
/22	1,024	1,022	Large campus
/16	65,536	65,534	Large organization
/8	16,777,216	16,777,214	ISP, very large org

Route Aggregation (Supernetting)

Suppose an ISP has been assigned `192.168.0.0/22` and allocates it to four customers:

```
Customer A: 192.168.0.0/24
Customer B: 192.168.1.0/24
Customer C: 192.168.2.0/24
Customer D: 192.168.3.0/24
```

To the rest of the internet, the ISP advertises a single entry: `192.168.0.0/22`. This covers all four /24s with one routing table entry. The ISP's internal routers know the longer-prefix entries for each customer; the global routing table sees only the /22.

CIDR Aggregation Principle

Hierarchical allocation allows hierarchical aggregation. If addresses are allocated in **aligned** blocks, they can always be summarized with a single shorter-prefix entry. CIDR reduced the global routing table from an explosive trajectory to a manageable one and bought the internet roughly a decade of additional IPv4 runway.

LPM and CIDR: The Connection

LPM + CIDR

The combination of CIDR and longest-prefix match (Lecture 13) is what makes the hierarchy work end-to-end. An ISP advertises `192.168.0.0/22`; a customer router adds a more specific `192.168.1.0/24` entry for one of its segments. LPM ensures that traffic *to* that segment uses the more specific entry, while all other traffic to the /22 uses the aggregate. Short prefixes aggregate; long prefixes specialize. Neither would work correctly without the other.

Demo: `ip addr show`

```
$ ip addr show
```

Sample output (Linux):

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether a4:c3:f0:12:34:56 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.42/24 brd 192.168.1.255 scope global dynamic eth0
    inet6 fe80::a6c3:f0ff:fe12:3456/64 scope link

3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether a4:c3:f0:78:9a:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.5.1/16 brd 10.0.255.255 scope global eth1
```

Reading the output:

- `127.0.0.1/8` — loopback interface; the entire `127.x.x.x` block routes locally; packets never leave the host
- `192.168.1.42/24` — host address `.42` on the `192.168.1.0/24` network; broadcast `.255` is shown explicitly
- `10.0.5.1/16` — host on a /16 network; broadcast is `10.0.255.255` (all host bits set)
- `fe80::.../64` — IPv6 link-local address (covered in Lecture 18)

macOS equivalent: `ifconfig`

4 Subnetting: Dividing Address Spaces

The Problem Subnetting Solves

An organization is assigned `192.168.10.0/24`—256 addresses, 254 usable. They have four departments (Engineering, Sales, HR, IT) and want to isolate them on separate network segments: different broadcast domains, different firewall rules, different DHCP scopes.

One option: obtain four separate /24 blocks from the ISP. This wastes ISP address space and inflates the ISP's routing table.

Better option: **subnet** the /24 into four smaller blocks internally. The ISP advertises one entry; the organization routes the pieces internally.

Subnetting Mechanics

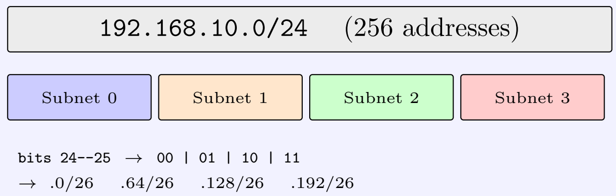
To create 2^k subnets from a `/N` block, **borrow k bits** from the host portion, extending the prefix to `/(N+k)`. Each subnet has $2^{(32-N-k)}$ addresses and $2^{(32-N-k)} - 2$ usable hosts.

Worked Example: Dividing 192.168.10.0/24 into Four /26s

Goal: divide `192.168.10.0/24` into 4 equal subnets for four departments.

Step 1: Choose k. $2^k = 4 \Rightarrow k = 2$. New prefix: $24 + 2 = 26$.

Step 2: Enumerate. Each /26 has $2^{32-26} = 64$ addresses, 62 usable.



Subnet	Network	Usable hosts	Broadcast
Engineering	`192.168.10.0/26`	`.1 -- .62`	`192.168.10.63`
Sales	`192.168.10.64/26`	`.65 -- .126`	`192.168.10.127`
HR	`192.168.10.128/26`	`.129 -- .190`	`192.168.10.191`
IT	`192.168.10.192/26`	`.193 -- .254`	`192.168.10.255`

Check: $4 \times 64 = 256$ total. The entire original /24 is accounted for; there is no waste and no overlap.

Subnet membership check. To determine whether two hosts share a subnet, AND both addresses with the /26 mask (`255.255.255.192`) and compare.

Are `192.168.10.75` and `192.168.10.100` in the same /26?

```
192.168.10.75  last octet: 01001011
192.168.10.100 last octet: 01100100
Mask /26                 : 11000000

.75  AND mask = 01000000  ->  192.168.10.64
.100 AND mask = 01000000  ->  192.168.10.64
```

Same network address $\Rightarrow$ **same subnet**. No router needed.

Are `192.168.10.75` and `192.168.10.130` in the same /26?

```
.75  AND mask = 01000000  ->  192.168.10.64
.130 AND mask = 10000000  ->  192.168.10.128
```

Different network addresses $\Rightarrow$ **different subnets**. Traffic must cross a router.

Variable-Length Subnet Masking (VLSM)

Subnets within an organization do not have to be equal-sized.[5] **VLSM** allows different prefix lengths in different parts of the same allocated block.

[5] Wikipedia: Variable-length subnet masking — VLSM, prefix-sized subnets, efficient allocation.

Example: BU ECE department, assigned `192.168.20.0/24`

Segment	Prefix	Usable hosts	Purpose
`192.168.20.0/25`	/25	126	Lab network
`192.168.20.128/26`	/26	62	Faculty offices
`192.168.20.192/27`	/27	30	Staff
`192.168.20.224/30`	/30	2	Router link

VLSM is efficient—blocks are sized to fit actual requirements—but requires careful tracking to avoid overlaps. Check: $128 + 64 + 32 + 4 = 228$ addresses allocated, leaving 28 for future use.

The Alignment Rule

A valid subnet must be **aligned**: the network address must be a multiple of the block size.

- A /26 has block size 64, so valid network addresses end in .0, .64, .128, .192
- `192.168.10.64/26` is valid: $64 = 1 \times 64$ ✓
- `192.168.10.10/26` is **invalid**: 10 is not a multiple of 64 ×

The alignment rule is why the bitwise AND with the mask works cleanly, and why any aligned block can be represented as a single prefix.

5 Special-Purpose IPv4 Address Ranges

RFC 1918 Private Addresses

RFC 1918 (1996)[6] designated three blocks as **private address space**—addresses that will never be assigned to hosts on the public internet:

Block	Range	Size	Typical use
`10.0.0.0/8`	10.0.0.0 – 10.255.255.255	16 M	Large enterprise, cloud VPCs
`172.16.0.0/12`	172.16.0.0 – 172.31.255.255	1 M	Medium enterprise
`192.168.0.0/16`	192.168.0.0 – 192.168.255.255	65 K	Home networks, small offices

Private addresses are:

- **Not routable on the public internet.** Routers at the internet boundary drop packets with private source or destination addresses.
- **Reusable.** Thousands of organizations can all use `192.168.1.0/24` without conflict, because these addresses are never visible outside their own networks.
- **Used behind NAT.** A single public IP can represent thousands of private hosts (covered in Lecture 18).

[6] Wikipedia: Private network — RFC 1918 address ranges, non-routable status, use with NAT.

Other Special-Purpose Ranges

Range	Description
`127.0.0.0/8`	**Loopback.**[a] `127.0.0.1` (localhost) always refers to the host itself; the entire /8 is loopback; packets never leave the host
`169.254.0.0/16`	**Link-local (APIPA).**[b] Automatically assigned when no DHCP server is found; not routable beyond the local link; if you see this on an interface, DHCP failed
`224.0.0.0/4`	**Multicast.**[c] Class D range; used for group communication (e.g., OSPF uses `224.0.0.5/.6`; RIP uses `224.0.0.9`)
`255.255.255.255`	**Limited broadcast.** Sent to all hosts on the local subnet; routers do not forward this
`100.64.0.0/10`	**Carrier-grade NAT (CGN, RFC 6598).**[d] Used by ISPs internally between their network and subscribers when RFC 1918 space runs out; may appear in `traceroute` output through certain ISPs
`0.0.0.0/0`	**Default route.** Matches anything not matched by a more specific entry
`0.0.0.0`	**"This host."** Used in DHCP Discover before the host has an address

[a]Wikipedia: Loopback — 127.0.0.0/8, localhost, loopback interface; packets never traverse a physical medium.
[b]Wikipedia: Link-local address — 169.254.0.0/16, APIPA (Automatic Private IP Addressing), IPv6 link-local.
[c]Wikipedia: Multicast address — 224.0.0.0/4, group communication, routing protocol multicast groups.
[d]Wikipedia: Carrier-grade NAT — 100.64.0.0/10, RFC 6598, ISP-level NAT as a stopgap for IPv4 exhaustion.

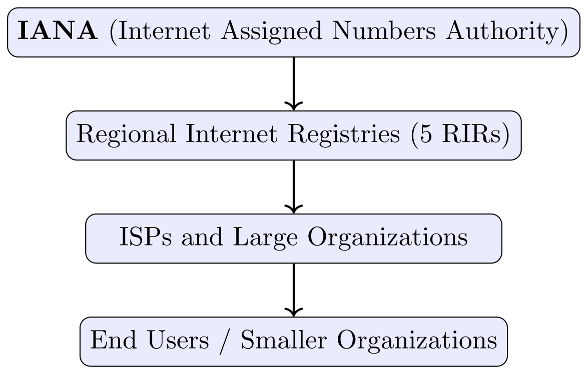
Practical encounters:

- `192.168.x.x` — your home router assigns these via DHCP; your laptop's address right now is probably in this range
- `10.x.x.x` — cloud provider VPCs (AWS, GCP, Azure) default to `10.0.0.0/8` blocks; you will configure these constantly
- `172.17.0.0/16` — Docker's default bridge network (a /12 subdivision); every container gets a `172.17.x.x` address by default
- `169.254.x.x` — seen on broken configurations; if you see this, DHCP failed
- `100.64.x.x` — may appear in `traceroute` through certain ISPs running CGN internally

6 Address Allocation Hierarchy and IPv4 Exhaustion

The Hierarchy

IPv4 addresses are allocated through a four-level hierarchy:[7] [8]



The five **Regional Internet Registries (RIRs)** each serve a geographic region:

RIR	Region
ARIN	North America
RIPE NCC	Europe, Middle East, Central Asia
APNIC	Asia-Pacific
LACNIC	Latin America and Caribbean
AFRINIC	Africa

ISPs subdivide their allocations and assign sub-blocks to customers. A residential ISP might assign a dynamic /32 (single address) to each subscriber via DHCP or PPP; a business ISP might assign a static /29 or /28 to a customer needing a few static IPs.

Organizations further subnet their allocated blocks for internal use, as practiced in Section 4.

IPv4 Exhaustion Timeline

The total IPv4 address space is $2^{32} \approx 4.3$ billion addresses.[9] By the mid-2000s, depletion was clearly imminent: some classful-era blocks had been allocated wastefully, and mobile device growth vastly exceeded 1981-era projections.

[7] Wikipedia: Regional Internet registry — ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC; allocation procedures.

[8] Wikipedia: Internet Assigned Numbers Authority — IANA, root of address allocation, coordination of protocol parameters.

[9] Wikipedia: IPv4 address exhaustion — exhaustion timeline, transfer markets, CGN as stopgap, IPv6 as long-term solution.

Event	Date
IANA exhausts free pool (last /8s allocated to RIRs)	February 2011
APNIC exhausts	April 2011
RIPE NCC exhausts	September 2012
LACNIC exhausts	June 2014
ARIN exhausts	September 2015
AFRINIC exhausts	2021

After exhaustion: RIRs entered waiting-list or transfer modes. New IPv4 addresses are no longer available from RIRs—organizations must buy them on the secondary market. IPv4 addresses now trade for approximately \$30–50 per address; a /24 block (256 addresses) is worth \$8,000–12,000.

What Kept IPv4 Alive: NAT

Network Address Translation (NAT)[10] was the stopgap that extended IPv4 life by roughly 20 years beyond original projections. One public IP address, shared by thousands of hosts behind a home or enterprise router, drastically reduced the number of public IPs needed. Without NAT, IPv6 would have been necessary much earlier. With NAT, the IPv4→IPv6 transition has been slow and uneven. NAT is examined in detail in Lecture 18.

IPv6: The Long-Term Answer

128-bit addresses: $2^{128} \approx 3.4 \times 10^{38}$—enough for every square millimeter of Earth's surface to have more addresses than the entire IPv4 internet. Deployment is ongoing but uneven (largely a mobile story, covered in Lecture 18).

[10] Wikipedia: Network address translation — NAT overview; how one public IP serves many private hosts; detail in Lecture 18.

7 Python Demo: The ipaddress Module

Python ipaddress Module

Python's **ipaddress** standard-library module (no install needed) provides a clean API for address and subnet arithmetic.

Basic network arithmetic:

```
import ipaddress

net = ipaddress.IPv4Network("192.168.10.0/24")

print(net.network_address)    # 192.168.10.0
print(net.broadcast_address)  # 192.168.10.255
print(net.netmask)            # 255.255.255.0
print(net.prefixlen)          # 24
print(net.num_addresses)      # 256

# Usable hosts (excludes network address and broadcast)
hosts = list(net.hosts())
print(f"First host: {hosts[0]}")    # 192.168.10.1
print(f"Last host:  {hosts[-1]}")   # 192.168.10.254
print(f"Usable:     {len(hosts)}")  # 254
```

Subnetting with .subnets():

```
# Divide 192.168.10.0/24 into four /26s
net = ipaddress.IPv4Network("192.168.10.0/24")
subnets = list(net.subnets(prefixlen_diff=2))  # add 2 bits -> /26

for i, s in enumerate(subnets):
    print(f"Subnet {i}: {s}  "
          f"hosts {s.network_address+1} - {s.broadcast_address-1}")
```

Output:

```
Subnet 0: 192.168.10.0/26   hosts 192.168.10.1   - 192.168.10.62
Subnet 1: 192.168.10.64/26  hosts 192.168.10.65  - 192.168.10.126
Subnet 2: 192.168.10.128/26 hosts 192.168.10.129 - 192.168.10.190
Subnet 3: 192.168.10.192/26 hosts 192.168.10.193 - 192.168.10.254
```

Containment checks:

```
# Is a host in a given subnet?
host = ipaddress.IPv4Address("192.168.10.75")
for s in subnets:
    if host in s:
        print(f"{host} is in {s}")
# -> 192.168.10.75 is in 192.168.10.64/26

# Are two hosts on the same subnet?
def same_subnet(a, b, prefixlen):
    net_a = ipaddress.IPv4Interface(f"{a}/{prefixlen}").network
    net_b = ipaddress.IPv4Interface(f"{b}/{prefixlen}").network
    return net_a == net_b

print(same_subnet("192.168.10.75",  "192.168.10.100", 26))  # True
print(same_subnet("192.168.10.75",  "192.168.10.130", 26))  # False
```

Special address checks:

8 Wireshark Demo: RFC 1918 vs. Public Addresses

Wireshark: RFC 1918 Addresses and the NAT Question

Open a capture of normal browser traffic. In the IP layer:

- **Source address**: your machine's RFC 1918 address (e.g., `192.168.1.42`)
- **Destination address**: the server's public IP (e.g., `142.250.80.46` — Google)
- **Return traffic**: source is the public IP; destination is your private IP

This immediately raises a question: if your source address is `192.168.1.42` (not routable on the internet), how does the server know where to send the reply?

Answer: NAT at your router rewrites the source address to the router's public IP before the packet reaches the internet. The router maintains a state table mapping `(private IP, port)` ↔ `(public IP, port)` so it can reverse the translation on return traffic. Covered in detail in Lecture 18.

Filter to highlight RFC 1918 sources:

```
ip.src == 192.168.0.0/16 || ip.src == 10.0.0.0/8
    || ip.src == 172.16.0.0/12
```

Observation: every packet your laptop sends has a private source address. Every reply comes back to that private address. Yet the packets traverse the public internet. This works only because your home router (or campus NAT gateway) is silently translating addresses on every outbound and inbound packet.

Discussion Questions

1. A company is assigned `172.20.0.0/16` and needs to create subnets for four buildings with 200, 150, 80, and 30 hosts respectively. Design a VLSM allocation. What prefix length do you choose for each building, and what are the network and broadcast addresses?
2. Your laptop shows address `169.254.23.45`. What does this tell you about the network configuration? What would you check first to diagnose the problem?
3. AWS by default gives a VPC the `10.0.0.0/16` block. When you create a subnet, you assign it a /24 within that range. What are you actually doing in terms of CIDR subnetting? Why does AWS reserve the first four addresses and the last address in each subnet (not just the network and broadcast addresses)?
4. A routing table entry for `0.0.0.0/0` is called the default route. Using what you know about longest-prefix match, explain precisely *why* this is a "default"—why does it match any packet that has no other match?
5. An ISP has been assigned `203.0.113.0/24` and wants to give each of eight customers a block of 16 addresses. What prefix length does each customer get? Write out all eight subnet network addresses. Does the alignment rule hold?

6. Before CIDR (in the classful era), the routing table had a separate entry for every Class A, B, and C network. Why did this cause a routing table explosion? Why didn't classful addressing allow aggregation?

7. You are examining a `traceroute` output and notice that several intermediate hops show RFC 1918 addresses (`10.x.x.x`). What does this tell you about the network topology? Is this surprising? (*Hint*: think about where the `traceroute` might be running from.)

8. IPv6 has 2^{128} addresses—enough that every device could have trillions of addresses. Yet IPv6 still uses prefix-based hierarchical allocation and longest-prefix match, just like IPv4. Why? Is prefix-based routing about address scarcity, or is something else driving it?

Further Reading

The following Wikipedia articles are the primary student-facing references for this lecture. They are suitable for reviewing concepts, exploring topics in greater depth, and following citations to original papers.

- Wikipedia: IPv4 — address structure, packet format, dotted-decimal notation, 32-bit address space.
- Wikipedia: Classful network — Class A/B/C/D/E, leading-bit encoding, historical allocation, and why classful addressing was abandoned in 1993.
- Wikipedia: Classless Inter-Domain Routing — CIDR, prefix notation, route aggregation, and RFC 1519 (1993).
- Wikipedia: Subnetwork — subnetting, subnet masks, how prefix length divides network and host portions.
- Wikipedia: Variable-length subnet masking — VLSM, mixed prefix lengths within a single address block.
- Wikipedia: Subnet mask — mask representation, bitwise AND with address, prefix-length equivalence.
- Wikipedia: Private network — RFC 1918 address ranges (`10/8`, `172.16/12`, `192.168/16`), non-routable status, use behind NAT.
- Wikipedia: Loopback — 127.0.0.0/8, localhost, loopback interface; packets never traverse a physical medium.
- Wikipedia: Link-local address — 169.254.0.0/16, APIPA (Automatic Private IP Addressing), IPv6 link-local.
- Wikipedia: Multicast address — 224.0.0.0/4, Class D, group communication, routing protocol multicast groups.
- Wikipedia: Carrier-grade NAT — 100.64.0.0/10, RFC 6598, ISP-level NAT as a stopgap for IPv4 exhaustion.
- Wikipedia: Regional Internet registry — ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC; allocation policies and procedures.

- Wikipedia: Internet Assigned Numbers Authority — IANA, root of the address allocation hierarchy, coordination of protocol parameters.
- Wikipedia: IPv4 address exhaustion — exhaustion timeline by RIR, transfer markets, CGN as stopgap, IPv6 as long-term solution.
- Wikipedia: Network address translation — NAT overview; one public IP serving many private hosts; detail in Lecture 18.
- Wikipedia: Longest prefix match — LPM algorithm, CIDR forwarding, priority among matching entries; covered in Lecture 13, applied throughout this lecture.

Prepared for EC 441, Boston University, Spring 2026.